

1. Package hota kya hai, aur kyun chahiye?

Soch ki tera ghar hai aur saara saamaan — kapde, bartan, kitaabein, tools — sab ek hi kamre mein dher laga ke rakha hai. Cheez dhundhna nightmare ho jayega na? Isliye hum almirah banate hain, kitchen alag, study table alag. **Package bhi yahi hai — code ko organize karne ka kamra/almirah.**

Transcript mein 4 fayde bataye gaye hain:

a) Code Reuse (DRY Principle — Don't Repeat Yourself) Agar ek hi kaam ka code tu 5 jagah copy-paste kar raha hai, woh bad practice hai. Pehla step: usko function mein daal do. Dusra step: agar woh function multiple files/projects mein chahiye, toh usko **package** mein daal do. Jaise tere Probus project mein email bhejne ka logic — woh ek hi jagah likha hai aur har campaign usko use karta hai. Wahi concept.

b) Naming Conflicts se bachna Agar packages na hote, toh poore project mein do functions ka same naam nahi ho sakta tha. Tujhe `sendEmailForCampaign`, `sendEmailForOTP` jaise lambe naam rakhne padte. Packages ke saath tu `email.Send()` aur `sms.Send()` likh sakta hai — dono ka naam `Send` hai, par alag-alag package mein hone ki wajah se koi conflict nahi. Jaise do alag buildings mein flat number 101 ho sakta hai — building ka naam disambiguate kar deta hai.

c) Better Organization Auth ka code auth folder mein, user ka code user folder mein. Naya developer aaye toh usko pata hai kahan kya milega.

d) Faster Compilation ⚡ Yeh interesting point hai. Go ka compiler **sirf un packages ko recompile karta hai jinmein change hua hai**. Agar tune sirf `auth` package mein change kiya, toh `user` package dobara compile nahi hoga — uska cached compiled version use ho jayega. Bade projects mein yeh compile time bahut bacha leta hai. (Isliye Go ka build itna fast feel hota hai compared to C++ jaisi languages.)

2. Module kya hai? (`go mod init`)

Yahan ek confusion common hai, toh clear kar deta hoon:

- **Module = tera poora project** (ek container)
- **Package = us project ke andar ka ek folder** (compartments)

Analogy: Module ek **building** hai, packages us building ke **flats** hain.

Project shuru karne ke liye:

```
bash
```

```
go mod init github.com/codersgyan/podcast
```

Yeh command ek `go.mod` file bana deti hai jisme do cheezein hoti hain:

1. Module ka naam (project ka identity)
2. Go ka version

Naam mein GitHub URL kyun? Yeh sirf ek **convention** hai, compulsory nahi. Tu `go mod init myapp` bhi likh sakta hai. Par GitHub URL isliye dete hain kyunki agar kal ko tu apna package open-source karega, toh log usko exactly usi path se `go get` karke install kar payenge. Naam globally unique ho jata hai.

(Purane zamaane mein `GOPATH` naam ka jhanjhat tha — ek specific folder mein hi projects banane padte the. Ab woh khatam, kahin bhi project bana sakte ho.)

3. Apna khud ka package banana

Video mein ek podcast app ka example hai. Login functionality multiple jagah use hogi (register ke baad, normal login, email verify ke baad), toh usko alag package mein rakha:

```
podcast/
├─ go.mod
├─ main.go      → package main
└─ auth/
    ├─ credentials.go → package auth
    └─ session.go     → package auth
```

Rules jo yaad rakhne hain:

1. **Package banana = bas ek folder banana** aur uske andar files mein top par `package auth` likhna. Bas. Koi config, koi registration nahi.
2. **Folder ka naam aur package ka naam same rakhna convention hai**, compulsory nahi. Par same rakho, warna log confuse honge.
3. **File ka naam kuch bhi ho sakta hai** — `credentials.go`, `session.go` — Go ko farak nahi padta. Jo matter karta hai woh file ke top ki `package auth` line hai.
4. **Ek folder ki saari files ek hi package ki hoti hain**. Matlab `credentials.go` mein bana function `session.go` mein bina import kiye directly use ho sakta hai — kyunki dono same package (same "kamra") mein hain.

`main` package special hai — woh entry point hota hai, usme `main()` function hota hai, wahin se program start hota hai.

4. Sabse IMPORTANT concept: Exported vs Unexported (Capital vs small letter) 🔑

Yeh is poore lesson ka heart hai. Go mein `export` keyword **nahi hai** (jaise JS/TS mein `export default` hota hai). Go ne ek bahut hi simple rule banaya:

Naam ka pehla letter Capital = Public (package ke bahar dikhega) Naam ka pehla letter small = private (sirf usi package ke andar)

```
go

package auth

// Capital L → exported → main.go mein auth.LoginWithCredentials() chalega
func LoginWithCredentials(username string, password string) {
    fmt.Println("Logging in user using", username, password)
}

// small e → unexported → sirf auth package ke andar use ho sakta hai
func extractSession() string {
    return "logged in"
}

// Public wrapper, private logic — common pattern
func GetSession() string {
    return extractSession()
}
```

Analogy: tera ghar. **Drawing room Capital hai** — mehmaan (dusre packages) aa sakte hain. **Bedroom small letter hai** — sirf ghar wale (same package) enter kar sakte hain.

Yeh `getSession` → `extractSession` wala pattern dhyan se dekh — **public API expose karo, internal logic private rakho**. Tere Probus codebase mein bhi yahi hota hoga: bahar se sirf clean functions dikhte hain, andar ka kachra (helper functions) hidden rehta hai. Isse tu kal internal logic badal sakta hai bina kisi caller ko tode — yahi encapsulation hai.

Struct fields par bhi yahi rule lagta hai! Yeh log bhool jaate hain:

```
go
package user

type User struct {
    Email string // Capital → bahar se accessible
    name  string // small → bahar se User.name nahi kar paoge! Compile error!
}
```

Toh agar tere struct ko JSON marshal karna hai ya dusre package mein use karna hai, fields Capital honi chahiye. (Isliye Go mein `json:"email"` tags lagte hain — kyunki field Capital `Email` hai par JSON mein lowercase chahiye.)

5. Apna package import kaise karte hain

```
go

package main

import (
    "fmt"
    "github.com/codersgyan/podcast/auth"
    "github.com/codersgyan/podcast/user"
)

func main() {
    auth.LoginWithCredentials("codersgyan", "secretpassword")

    session := auth.GetSession()
    fmt.Println(session)

    u := user.User{
        Email: "user@email.com",
        Name:  "John Doe",
    }
    fmt.Println(u.Email, u.Name)
}
```

Import path ka formula: **module ka naam + folder ka path**. Module =

`github.com/codersgyan/podcast`, folder = `auth`, toh import =
`github.com/codersgyan/podcast/auth`.

Run karne ke liye: `go run main.go` — output: `Logging in user using codersgyan secretpassword`.

Ek chhoti baat video mein aayi: package ka naam choose karte waqt dhyan rakho ki standard library se clash na ho — `user` naam ka OS-related package bhi hota hai, toh editor mein kabhi galat autocomplete aa sakta hai.



6. Third-party packages (`go get`)

Jaise JS/TS mein npm se `express` install karte ho, Go mein bhi community ke banaye packages use kar sakte ho. Haan, Go ki standard library itni rich hai ki zaroorat kam padti hai (HTTP server tak built-in hai!), par jab chahiye:

Example: `fatih/color` package — terminal output ko coloured banane ke liye.

```
bash

go get github.com/fatih/color
```

Phir code mein:

```
go

color.Red("User email: %s", u.Email)
color.Green("Success!")
```

Aur output terminal mein red/green colour mein aayega. CLI tools banate waqt yeh kaafi useful hai.

7. `go.mod` mein `// indirect` ka matlab

Jab tune `color` install kiya, `go.mod` ke `require` section mein **4 packages** aa gaye, ek nahi. Kyun? Kyunki `color` khud 3 aur packages par depend karta hai. Woh automatically install ho gaye.

- **Direct dependency** = jo TU apne code mein import karta hai (`color`)
- **Indirect dependency** = jo tere dependencies ke dependencies hain (`// indirect` comment ke saath)

Analogy: tune Swiggy se pizza order kiya (direct). Pizza banane ke liye restaurant ne cheese supplier se cheese mangaya (indirect) — tujhe cheese supplier se directly matlab nahi, par woh chain ka hissa hai.

Ek aur baat: jab tak tu installed package ko code mein actually **use nahi karta**, Go usko `// indirect` hi dikhata hai — kyunki compiler ko pata hai ki abhi koi direct import nahi hai.

8. `go.sum` file

Yeh lock file hai — bilkul `package-lock.json` jaisi (npm world se tu familiar hai). Isme har dependency ka **checksum** (cryptographic hash) hota hai. Fayda: jab tu ya Harshi ya CI server project install kare, sabko **exactly same version, same bytes** milein. Koi beech mein package tamper kar de toh checksum match nahi hoga aur Go error de dega. Consistency + security.

9. `go mod tidy` — the cleanup magic ✨

Yeh bahut powerful command hai, production projects mein roz use hoti hai:

```
bash
go mod tidy
```

Yeh teen kaam karti hai:

1. Jo packages tu code mein use kar raha hai par install nahi hain → install kar deti hai. (Matlab tu bina `go get` kiye direct import line likh de, phir `go mod tidy` chala de — ho gayal!)
2. Jo install hain par use nahi ho rahe → hata deti hai.
3. Direct vs indirect ko sahi se organize kar deti hai `go.mod` mein.

Soch isko ek "kamre ki safai" command ki tarah — jo chahiye woh laao, jo nahi chahiye woh feko, sab cheez sahi shelf par rakho.



Quick Recap — 3 cheezein jo video chahta hai tu yaad rakhe

1. **Capital letter = exported (public), small letter = unexported (private)** — functions, structs, struct fields, variables, sab par lagta hai. Yeh Go ka visibility system hai, koi `export` keyword nahi.
2. **Third-party packages** `go get <url>` se install hote hain, import path module-name convention follow karta hai.

3. **go mod tidy** dependencies ko auto-fix/install/clean karta hai. **go.sum** versions ko lock karke consistency deta hai.
-

Ek personal note: yeh sab tere Probus codebase mein already live hai — tera notification service ke folders (jaise email, campaign, queue waghera) sab packages hi hain, aur jo functions tu handlers se call karta hai woh sab Capital letter se start hote honge. Kal office mein ek baar codebase kholke conscious nazar se dekh: kaunse functions Capital hain, kaunse small, aur kyun. Tab yeh lesson "video knowledge" se "apna knowledge" ban jayega.

Koi specific part — jaise import path resolution ya **indirect** wala part — aur deep chahiye toh bata, ya tu apni analogy bana ke verify karwana chahe toh woh bhi chalega. 💪